Java Loops — 30 Programs

Beginner to Intermediate • Interview-Oriented • Code + Output

What this workbook covers

for loop, while loop, do-while loop, enhanced for loop, break, continue, nested loops, number problems, patterns, strings, arrays, and common interview practice.

1. For Loop — Print 1 to 10

Java Code:

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++)
            System.out.print(i + " ");
    }
}
```

Output:

```
1 2 3 4 5 6 7 8 9 10
```

2. For Loop — Print 10 to 1

Java Code:

```
public class Main {
    public static void main(String[] args) {
        for (int i = 10; i >= 1; i--)
            System.out.print(i + " ");
    }
}
```

Output:

```
10 9 8 7 6 5 4 3 2 1
```

3. Even Numbers from 1 to 20

Java Code:

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 20; i++) {
            if (i % 2 == 0)
                System.out.print(i + " ");
        }
    }
}
```

Output:

```
2 4 6 8 10 12 14 16 18 20
```

4. Odd Numbers from 1 to 20

Java Code:

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 20; i++) {
            if (i % 2 != 0)
                System.out.print(i + " ");
        }
    }
}
```

Output:

```
1 3 5 7 9 11 13 15 17 19
```

5. Sum of Numbers 1 to 10

Java Code:

```
public class Main {
    public static void main(String[] args) {
        int sum = 0;
        for (int i = 1; i <= 10; i++)
            sum = sum + i;
        System.out.println("Sum = " + sum);
    }
}
```

Output:

```
Sum = 55
```

6. Multiplication Table of 5

Java Code:

```
public class Main {
    public static void main(String[] args) {
        int n = 5;
        for (int i = 1; i <= 10; i++)
            System.out.println(n + " x " + i + " = " + n * i);
    }
}
```

Output:

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

7. Factorial of a Number

Java Code:

```
public class Main {
    public static void main(String[] args) {
        int n = 5, factorial = 1;
        for (int i = 1; i <= n; i++)
            factorial *= i;
        System.out.println("Factorial = " + factorial);
    }
}
```

Output:

```
Factorial = 120
```

8. Count Digits using while

Java Code:

```
public class Main {
    public static void main(String[] args) {
        int n = 12345, count = 0;
        while (n != 0) {
            n /= 10;
            count++;
        }
        System.out.println("Digits = " + count);
    }
}
```

Output:

```
Digits = 5
```

9. Reverse a Number

Java Code:

```
public class Main {
    public static void main(String[] args) {
        int n = 12345, reverse = 0;
        while (n != 0) {
            int digit = n % 10;
            reverse = reverse * 10 + digit;
            n /= 10;
        }
        System.out.println("Reverse = " + reverse);
    }
}
```

Output:

```
Reverse = 54321
```

10. Sum of Digits

Java Code:

```
public class Main {
    public static void main(String[] args) {
        int n = 12345, sum = 0;
        while (n != 0) {
            sum += n % 10;
            n /= 10;
        }
        System.out.println("Digit sum = " + sum);
    }
}
```

Output:

```
Digit sum = 15
```

11. Palindrome Number

Java Code:

```
public class Main {
    public static void main(String[] args) {
        int n = 121, original = n, reverse = 0;
        while (n != 0) {
            reverse = reverse * 10 + n % 10;
            n /= 10;
        }
        System.out.println(original == reverse ? "Palindrome" : "Not Palindrome");
    }
}
```

Output:

```
Palindrome
```

12. Armstrong Number

Java Code:

```
public class Main {
    public static void main(String[] args) {
        int n = 153, original = n, sum = 0;
        while (n != 0) {
            int digit = n % 10;
            sum += digit * digit * digit;
            n /= 10;
        }
        System.out.println(sum == original ? "Armstrong Number" : "Not Armstrong");
    }
}
```

Output:

```
Armstrong Number
```

13. Prime Number Check

Java Code:

```
public class Main {
    public static void main(String[] args) {
        int n = 29;
        boolean prime = n > 1;

        for (int i = 2; i < n && prime; i++) {
            if (n % i == 0)
                prime = false;
        }

        System.out.println(prime ? "Prime" : "Not Prime");
    }
}
```

Output:

```
Prime
```

14. Prime Numbers from 1 to 50

Java Code:

```
public class Main {
    public static void main(String[] args) {
        for (int n = 2; n <= 50; n++) {
            boolean prime = true;

            for (int i = 2; i < n; i++) {
                if (n % i == 0) {
                    prime = false;
                    break;
                }
            }

            if (prime)
                System.out.print(n + " ");
        }
    }
}
```

Output:

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

15. Fibonacci Series

Java Code:

```
public class Main {
    public static void main(String[] args) {
        int a = 0, b = 1;

        for (int i = 1; i <= 10; i++) {
            System.out.print(a + " ");
            int c = a + b;
            a = b;
            b = c;
        }
    }
}
```

Output:

```
0 1 1 2 3 5 8 13 21 34
```

16. While Loop — Print 1 to 10

Java Code:

```
public class Main {
    public static void main(String[] args) {
        int i = 1;

        while (i <= 10) {
            System.out.print(i + " ");
            i++;
        }
    }
}
```

Output:

```
1 2 3 4 5 6 7 8 9 10
```

17. Do-While Loop

Java Code:

```
public class Main {
    public static void main(String[] args) {
        int i = 1;

        do {
            System.out.print(i + " ");
            i++;
        } while (i <= 5);
    }
}
```

Output:

```
1 2 3 4 5
```

18. Do-While Executes at Least Once

Java Code:

```
public class Main {
    public static void main(String[] args) {
        int i = 10;

        do {
            System.out.println("Executed once");
            i++;
        } while (i < 5);
    }
}
```

Output:

```
Executed once
```

19. break Statement

Java Code:

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i == 6)
                break;
            System.out.print(i + " ");
        }
    }
}
```

Output:

```
1 2 3 4 5
```

20. continue Statement

Java Code:

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i == 5)
                continue;
            System.out.print(i + " ");
        }
    }
}
```

Output:

```
1 2 3 4 6 7 8 9 10
```

21. Nested Loop — Rectangle Pattern

Java Code:

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 5; j++)
                System.out.print("* ");
            System.out.println();
        }
    }
}
```

Output:

```
* * * * *
* * * * *
* * * * *
```

22. Right Triangle Star Pattern

Java Code:

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            for (int j = 1; j <= i; j++)
                System.out.print("* ");
            System.out.println();
        }
    }
}
```

Output:

```
*
* *
* * *
* * * *
* * * * *
```

23. Number Triangle

Java Code:

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            for (int j = 1; j <= i; j++)
                System.out.print(j + " ");
            System.out.println();
        }
    }
}
```

Output:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

24. Reverse Star Triangle

Java Code:

```
public class Main {
    public static void main(String[] args) {
        for (int i = 5; i >= 1; i--) {
            for (int j = 1; j <= i; j++)
                System.out.print("* ");
            System.out.println();
        }
    }
}
```

Output:

```
* * * * *
* * * *
* * *
* *
*
```

25. Nested Multiplication Tables 2 to 5

Java Code:

```
public class Main {
    public static void main(String[] args) {
        for (int n = 2; n <= 5; n++) {
            System.out.println("Table of " + n);

            for (int i = 1; i <= 5; i++)
                System.out.println(n + " x " + i + " = " + n * i);
        }
    }
}
```

Output:

```
Table of 2
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
Table of 3
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
Table of 4
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
Table of 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
```

26. Count Vowels in a String

Java Code:

```
public class Main {
    public static void main(String[] args) {
        String str = "Java Programming";
        int count = 0;

        for (int i = 0; i < str.length(); i++) {
            char ch = Character.toLowerCase(str.charAt(i));

            if (ch == 'a' || ch == 'e' || ch == 'i' ||
                ch == 'o' || ch == 'u')
                count++;
        }

        System.out.println("Vowels = " + count);
    }
}
```

Output:

```
Vowels = 5
```

27. Reverse a String using Loop

Java Code:

```
public class Main {
    public static void main(String[] args) {
        String str = "Java";
        String reverse = "";

        for (int i = str.length() - 1; i >= 0; i--)
            reverse += str.charAt(i);

        System.out.println("Reverse = " + reverse);
    }
}
```

Output:

```
Reverse = avaJ
```

28. Find Largest Array Element

Java Code:

```
public class Main {
    public static void main(String[] args) {
        int[] numbers = {10, 45, 23, 89, 12};
        int largest = numbers[0];

        for (int i = 1; i < numbers.length; i++) {
            if (numbers[i] > largest)
                largest = numbers[i];
        }

        System.out.println("Largest = " + largest);
    }
}
```

Output:

```
Largest = 89
```

29. Sum Array using Enhanced for Loop

Java Code:

```
public class Main {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};
        int sum = 0;

        for (int number : numbers)
            sum += number;

        System.out.println("Sum = " + sum);
    }
}
```

Output:

```
Sum = 150
```

30. Count Positive, Negative and Zero

Java Code:

```
public class Main {
    public static void main(String[] args) {
        int[] numbers = {10, -5, 0, 20, -8, 0, 15};

        int positive = 0, negative = 0, zero = 0;

        for (int number : numbers) {
            if (number > 0)
                positive++;
            else if (number < 0)
                negative++;
            else
                zero++;
        }

        System.out.println("Positive = " + positive);
        System.out.println("Negative = " + negative);
        System.out.println("Zero = " + zero);
    }
}
```

Output:

```
Positive = 3
Negative = 2
Zero = 2
```

Quick Loop Revision

1. **for loop** — Best when you know how many times the loop should run.
2. **while loop** — Condition is checked before every iteration.
3. **do-while loop** — The body executes at least once because the condition is checked after the body.
4. **enhanced for loop** — Simple way to traverse arrays and collections.
5. **break** — Immediately exits the loop.
6. **continue** — Skips the current iteration and moves to the next one.
7. **nested loop** — A loop inside another loop; commonly used for patterns and tables.

Important interview problems to practice:

Reverse number • Palindrome • Prime • Factorial • Fibonacci • Armstrong • Digit sum • Star patterns • Reverse String • Largest array element • Enhanced for loop • break and continue

Memory Tip: for = known repetitions, while = condition-controlled repetition, do-while = runs at least once.